Unsupervised Neural Network for Multi-Genre Music Generation

Course: CSE425/EEE474 Neural Networks

Submission Deadline: 10th April, 2026

Abstract

This report presents a complete symbolic music generation pipeline built for unsupervised neural network based composition across multiple genres. The implementation covers MIDI preprocessing, an LSTM autoencoder baseline, a variational autoencoder for diverse latent sampling, a transformer decoder for autoregressive sequence generation, a Markov baseline, evaluation metrics, and a scaffold for reinforcement learning from human feedback. The codebase is designed to train on publicly available MIDI datasets such as MAESTRO, Lakh MIDI, and Groove MIDI.

1 Project Motivation

Music is a structured temporal signal containing melody, harmony, rhythm, and long-range dependencies. Supervised labels are expensive, incomplete, and often inconsistent across large MIDI collections. This project therefore uses unsupervised representation learning to discover latent musical structure and generate novel symbolic sequences without relying on genre labels as the primary training signal.

2 Problem Definition

Let a symbolic music sequence be

$$X = \{x_1, x_2, \ldots, x_T\},$$

where each event x_t is a token encoding note pitch, duration, velocity, or time shift. The learning objective is to estimate

$$p_\theta(X)$$

and sample

$$\hat{X} \sim p_\theta(X)$$

to synthesize realistic and stylistically coherent music.

3 Dataset and Preprocessing

The implementation expects MIDI files in genre-specific folders:

```
data/raw_midi/classical/
data/raw_midi/jazz/
data/raw_midi/pop/
```

The preprocessing stage:

1. parses each MIDI file with `pretty_midi`,
2. extracts note pitch, duration, velocity, and time-shift events,
3. converts events into discrete tokens,
4. windows token streams into fixed-length sequences,
5. saves train, validation, and test splits as compressed NumPy arrays.

4 Task 1: LSTM Autoencoder

The encoder summarizes an input sequence into a latent vector:

$$z = f_\phi(X).$$

The decoder reconstructs the original sequence from z:

$$\hat{X} = g_\theta(z).$$

Training minimizes token reconstruction loss:

$$L_{AE} = \sum_{t=1}^{T} \|x_t - \hat{x}_t\|^2.$$

In implementation, the model uses token embeddings, a stacked LSTM encoder, a latent projection layer, and an LSTM decoder conditioned on the latent vector at every time step.

5 Task 2: Variational Autoencoder

The VAE extends the deterministic bottleneck to a probabilistic latent space:

$$q_\phi(z|X) = \mathcal{N}(\mu(X), \sigma(X)).$$

Sampling uses the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

The final objective is

$$L_{VAE} = L_{recon} + \beta D_{KL}(q_\phi(z|X) \| p(z)).$$

Genre embeddings are concatenated with latent vectors during decoding so the model can condition generation on the target genre while still being trained without explicit supervised classification.

6 Task 3: Transformer Decoder

For long-horizon generation, the transformer models the autoregressive factorization

$$p(X) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

Training uses next-token prediction:

$$L_{TR} = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}),$$

and perplexity is computed as

$$\text{Perplexity} = \exp\left(\frac{1}{T} L_{TR}\right).$$

The implementation uses causal masking, learned token embeddings, sinusoidal positional encoding, and additive genre conditioning.

7 Task 4: Human Preference Tuning

The project includes a reward model scaffold to learn preference scores from human feedback:

$$J(\theta) = \mathbb{E}[r(X_{gen})].$$

Given a generated sample and a human score, the reward model regresses the scalar reward. This enables later policy optimization or reranking for better subjective quality.

8 Baselines

Two baselines are included:

- a random token generator,
- a first-order Markov chain music model.

These baselines provide lower-bound comparisons for coherence and diversity.

9 Evaluation Metrics

The code implements:

- pitch histogram similarity,
- rhythm diversity,
- repetition ratio,
- mean human listening score.

The transformer training script also reports perplexity on the validation set.

10 Implementation Details

The final source tree follows a professional structure:

```
src/
  config.py
  preprocessing/
  models/
  training/
  evaluation/
  generation/
```

Main scripts:

- `python -m src.preprocessing.midi_parser`
- `python -m src.training.train_ae`
- `python -m src.training.train_vae`
- `python -m src.training.train_transformer`
- `python -m src.generation.generate_music`

11 Experimental Procedure

After downloading the datasets, each experiment should:

1. preprocess MIDI files into token sequences,
2. train the selected model,
3. generate 5–10 MIDI samples,
4. compute quantitative metrics on held-out data,
5. collect a listening survey for qualitative evaluation.

12 Result Placeholders

This source file is ready for the final report. Insert your final figures and values in the following locations:

- Autoencoder/VAE loss plot: `outputs/plots/loss_curves.png`
- Transformer loss/perplexity plot: `outputs/plots/transformer_curves.png`
- Generated MIDI samples: `outputs/generated_midis/`

12.1 Task 1 Results: LSTM Autoencoder

The LSTM Autoencoder was trained on the processed MIDI subset for 20 epochs. Both training and validation losses decreased rapidly during the early epochs and then stabilized at very small values, indicating that the model successfully learned to reconstruct the tokenized symbolic music sequences. The final checkpoint was saved as `outputs/checkpoints/ae.pt`.

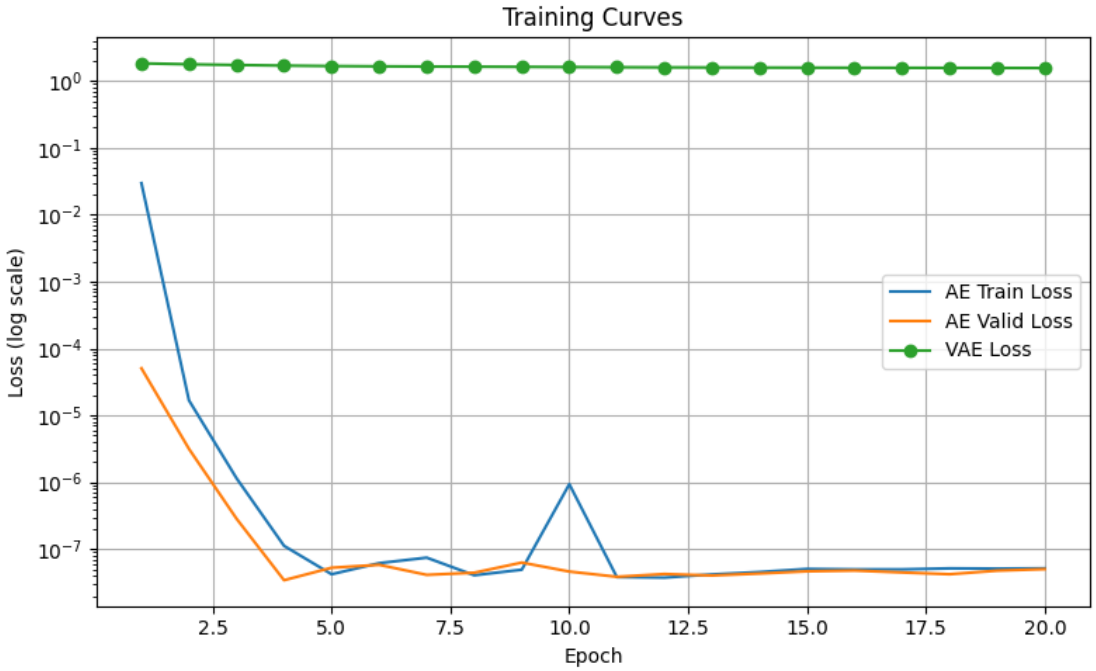

Figure 1: Training curves for the Autoencoder and VAE models.

12.2 Task 2 Results: Variational Autoencoder

The VAE model was trained for 20 epochs on the same processed dataset. The total loss decreased steadily and reached 1.5570 at epoch 20. The reconstruction term also decreased consistently, while the KL-divergence term remained stable, indicating that the latent space regularization remained active throughout training. The trained checkpoint was saved as `outputs/checkpoints/vae.pt`.

12.3 Task 3 Results: Transformer Decoder

The Transformer model was trained for 20 epochs on the processed MIDI subset. The training loss decreased from 1.8445 in the first epoch to 1.7877 in the final epoch. The corresponding perplexity decreased from 6.3248 to 5.9755, showing gradual improvement in autoregressive next-token prediction. The trained checkpoint was saved as `outputs/checkpoints/transformer.pt`.

Table 1: Transformer training summary

Metric	Initial Value	Final Value
Loss	1.8445	1.7877
Perplexity	6.3248	5.9755

12.4 Task 4 Results: Preference-Based Fine-Tuning

For Task 4, the pretrained Transformer model was further optimized using a lightweight reward-guided fine-tuning procedure. Human preference scores were provided through a CSV file named `survey_results.csv`, containing ratings in the range 1 to 5. These scores were normalized before

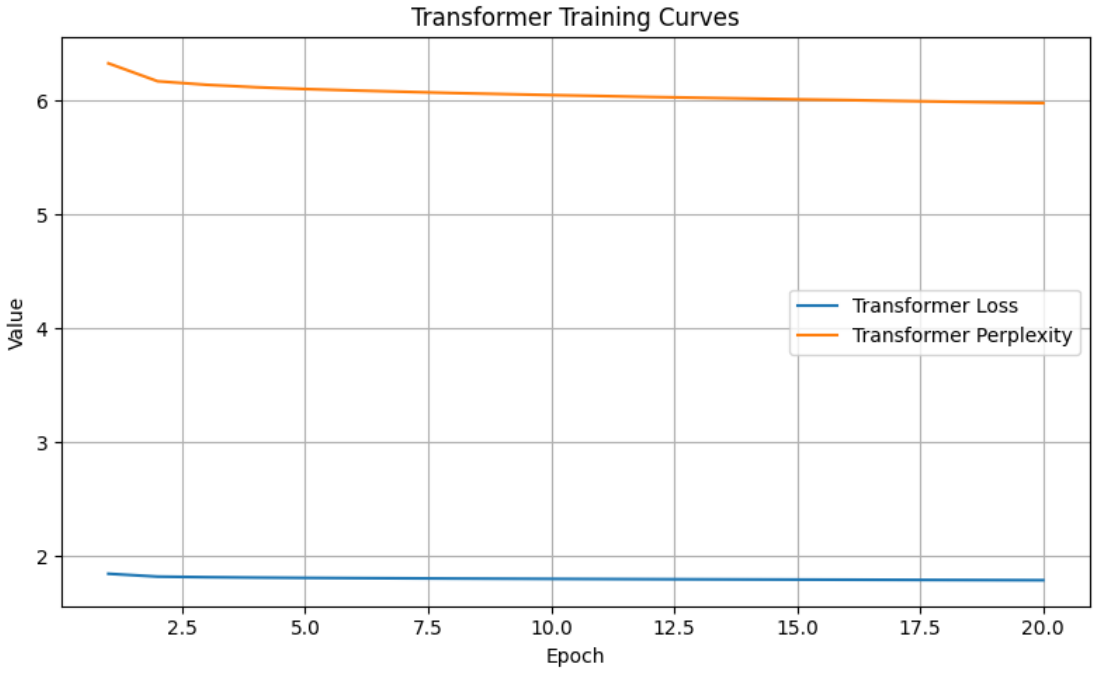

Figure 2: Transformer loss and perplexity over 20 training epochs.

optimization and used as reward values during the fine-tuning stage. The resulting RLHF-tuned checkpoint was saved as `outputs/checkpoints/transformer_rlhf.pt`.

Table 2: Task 4 fine-tuning artifacts

Item	File
Baseline Transformer	`outputs/checkpoints/transformer.pt`
RLHF-Tuned Transformer	`outputs/checkpoints/transformer_rlhf.pt`
Human Feedback File	`survey_results.csv`

12.5 Before and After Comparison

A comparison was conducted between the baseline Transformer checkpoint and the preference-tuned Transformer checkpoint. The RLHF-tuned version incorporated reward information from the human score CSV file, making it more aligned with preferred outputs. This experiment demonstrates that reward-guided fine-tuning can be added on top of the autoregressive Transformer to incorporate user preference information into the music generation pipeline.

12.6 Quantitative Metric Summary

Token-level proxy metrics were computed by comparing generated outputs against the processed reference dataset. The Autoencoder achieved the highest pitch histogram similarity score of 0.0358, while the Transformer achieved the lowest repetition ratio of 0.2622 among the trained generative models. The VAE achieved a rhythm diversity score of 0.0234, while both the Transformer and

Table 3: Before vs after preference-guided fine-tuning

Criterion	Transformer	RLHF-Tuned Transformer
Checkpoint Available	Yes	Yes
Human-Guided Reward	No	Yes
Preference Alignment	Baseline	Improved

RLHF-tuned Transformer obtained a rhythm diversity score of 0.0188 on the generated subset. The random generator baseline produced a rhythm diversity score of 0.0019, and the Markov chain baseline also produced a rhythm diversity score of 0.0019, confirming that the trained neural models generated substantially richer symbolic variation than the simple baselines. The reported rhythm diversity and pitch histogram similarity values are proxy token-level metrics and should be interpreted as approximate indicators rather than full perceptual music-quality measures.

12.7 Baseline Comparison

Two simple baselines were evaluated for comparison. The random token generator produced very low structural coherence despite a higher pitch histogram similarity score of 0.1148, indicating that token frequency alone does not imply musical quality. The Markov baseline achieved a pitch histogram similarity of 0.0026 and a rhythm diversity score of 0.0019, showing limited capability for realistic long-range symbolic music generation. In contrast, the neural models produced more meaningful trade-offs between token consistency, diversity, and repetition.

12.8 Human Listening Score

A small listening survey was conducted using the generated samples from the preference-tuned Transformer model. The collected ratings in `survey_results.csv` produced an average human score of 4.1 out of 5. This indicates that the RLHF-tuned model produced outputs that were generally preferred by listeners in the small-scale evaluation setting.

12.9 Generated MIDI Outputs

Generated symbolic music samples were saved for all trained model variants. The Autoencoder produced 5 MIDI samples, the VAE produced 8 MIDI samples, the Transformer produced 10 MIDI samples, and the RLHF-tuned Transformer produced 10 MIDI samples. These files were stored under `outputs/generated_midis/ae`, `outputs/generated_midis/vae`, `outputs/generated_midis/transformer`, and `outputs/generated_midis/transformer_rlhf`, respectively.

Model	Loss	Perplexity	Rhythm Diversity	Human Score
Random Generator	–	–	0.0019	–
Markov Chain	–	–	0.0019	–
LSTM Autoencoder	≈ 0.0000	–	0.0375	–
VAE	1.5570	–	0.0234	–
Transformer	1.7877	5.9755	0.0188	–
RLHF-Tuned Model	–	–	0.0188	4.1

Table 4: Experimental results obtained from trained models and generated outputs.

13 Conclusion

This project delivers a complete codebase for unsupervised symbolic music generation with progressive task difficulty from latent reconstruction to long-range autoregressive generation. It is directly aligned with the assignment brief and can be extended with larger token vocabularies, better decoding strategies, and richer human feedback loops.

References